

现代编程思想

泛型、高阶函数与迭代器

月兔公开课课程组

设计良好的抽象

- 软件工程中，我们要设计良好的抽象
 - 当代码多次重复出现
 - 当抽出的逻辑具有合适的语义
- 编程语言为我们提供了各种抽象的手段
 - 函数、泛型、高阶函数、接口.....

泛型函数与泛型数据

堆栈

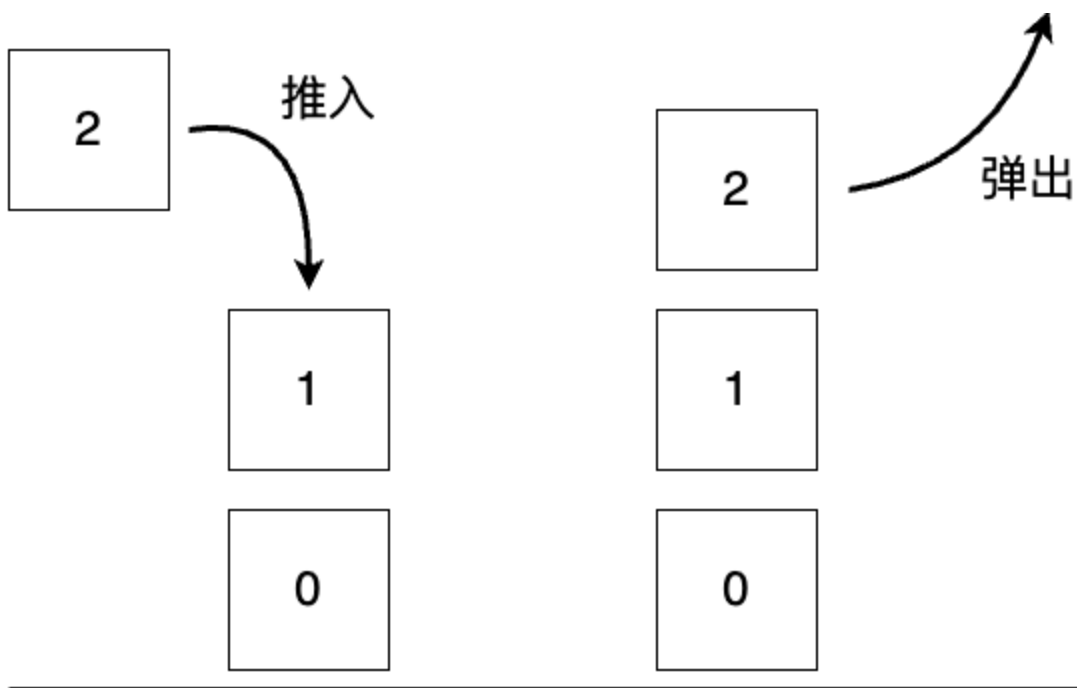
- 栈是一个由一系列对象组成的一个集合，这些对象的插入和删除遵循**后进先出**原则 (Last In First Out)



堆栈

- 我们定义以下操作，以存储整数的堆栈 `IntStack` 为例

```
1. fn empty() -> IntStack { ... } // 创建新的堆栈  
2. fn push(e : Int, stack : IntStack) -> IntStack { ... } // 将新的元素加入栈顶  
3. fn pop(stack : IntStack) -> (Option[Int], IntStack) { ... } // 从堆栈取出元素
```



整数堆栈

- 我们实现整数堆栈的定义
 - `self` 关键字允许我们链式调用: `IntStack::empty().push(1).pop()`

```

1. enum IntStack {
2.     Empty
3.     NonEmpty(Int, IntStack)
4. }
5. fn IntStack::empty() -> IntStack { Empty }
6. fn IntStack::push(self : IntStack, value : Int) -> IntStack { NonEmpty(value, self) }
7. fn IntStack::pop(self : IntStack) -> (Option[Int], IntStack) {
8.     match self {
9.         Empty => (None, Empty)
10.        NonEmpty(top, rest) => (Some(top), rest)
11.    }
12. }

```

- 之前定义列表就是一个堆栈

字符串堆栈

- 除了存储整数，我们也会希望存储字符串

```
1. enum StringStack {  
2.     Empty  
3.     NonEmpty(String, StringStack)  
4. }  
5. fn StringStack::empty() -> StringStack { Empty }  
6. fn StringStack::push(self : StringStack, value : String) -> StringStack { NonEmpty(value, self) }  
7. fn StringStack::pop(self : StringStack) -> (Option[String], StringStack) {  
8.     match self {  
9.         Empty => (None, Empty)  
10.        NonEmpty(top, rest) => (Some(top), rest)  
11.    }  
12. }
```

- 我们希望存储很多很多类型在堆栈中
 - 每个类型都要定义一个对应的堆栈吗？
 - `IntStack` 和 `StringStack` 似乎结构一模一样？

泛型数据结构与泛型函数

- 泛型数据结构与泛型函数以类型为参数，构建更抽象的结构

```
1. enum Stack[T] {  
2.     Empty  
3.     NonEmpty(T, Stack[T])  
4. }  
5. fn[T] Stack::empty() -> Stack[T] { Empty }  
6. fn[T] Stack::push(self : Stack[T], value : T) -> Stack[T] { NonEmpty(value, self) }  
7. fn[T] Stack::pop(self : Stack[T]) -> (Option[T], Stack[T]) {  
8.     match self {  
9.         Empty => (None, Empty)  
10.        NonEmpty(top, rest) => (Some(top), rest)  
11.    }  
12. }
```

- 将 `T` 替换为 `Int` 或 `String` 即相当于 `IntStack` 与 `StringStack`

泛型数据结构与泛型函数

- 我们用 `[<类型1>, <类型2>, ...]` 来定义泛型的类型参数
 - `enum Stack[T]{ Empty; NonEmpty(T, Stack[T]) }`
 - `struct Pair[A, B]{ first : A; second : B }`
 - `fn identity[A](value : A) -> A { value }`
 - `Stack` 与 `Pair` 可以看做从类型上的函数：类型构造器
- 类型参数多数时候会根据参数被自动推导

```
let empty = Stack::empty()
```

```
let one = empty.push(1)
```

```
let two = one.push(1)
```

```
let (top, rest) = two.pop()
```

```
(Stack[Int]) -> (Option[Int], Stack[Int])
```

泛型数据结构：队列

- 我们定义如下的操作：

```
1. fn[T] Queue::empty() -> Queue[T] { ... } // 创建空队列
2. fn[T] Queue::push(q : Queue[T], x : T) -> Queue[T] { ... } // 向队尾添加元素
3. // 尝试取出一个元素，并返回剩余队列；若为空则为本身
4. fn[T] Queue::pop(q : Queue[T]) -> (Option[T], Queue[T]) { ... }
```

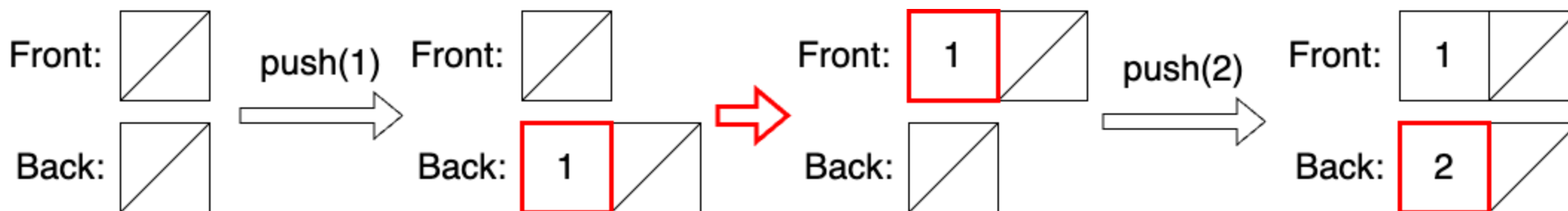
- 我们可以用一个列表（堆栈）模拟队列，但是效率低下
 - 在队尾添加元素需要重新构建整个列表
 - `Cons(1, Cons(2, Nil)) => Cons(1, Cons(2, Cons(3, Nil)))`

泛型数据结构：队列

- 我们用两个堆栈模拟队列

```
1. struct Queue[T] {  
2.     front : Stack[T] // 负责取出操作  
3.     back  : Stack[T] // 负责存储操作  
4. }
```

- 当添加数据时，存入 `back`；当读取数据时，从 `front` 中取出
- 操作后，若 `front` 为空，则通过反转队列，将 `back` 转为 `front`
 - 确保若队列非空，则 `front` 非空
 - 队列反转的开销将在多次读取中平摊

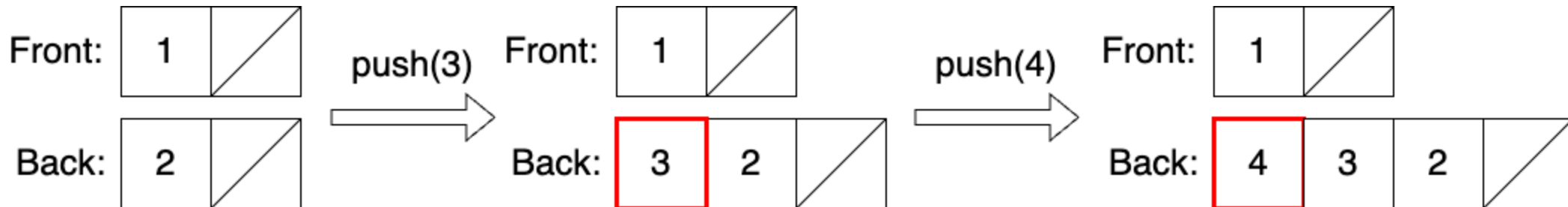


泛型数据结构：队列

- 我们用两个堆栈模拟队列

```
1. struct Queue[T] {  
2.     front : Stack[T] // 负责取出操作  
3.     back  : Stack[T] // 负责存储操作  
4. }
```

- 当添加数据时，存入 `back`；当读取数据时，从 `front` 中取出
- 操作后，若 `front` 为空，则通过反转队列，将 `back` 转为 `front`
 - 确保若队列非空，则 `front` 非空
 - 队列反转的开销将在多次读取中平摊

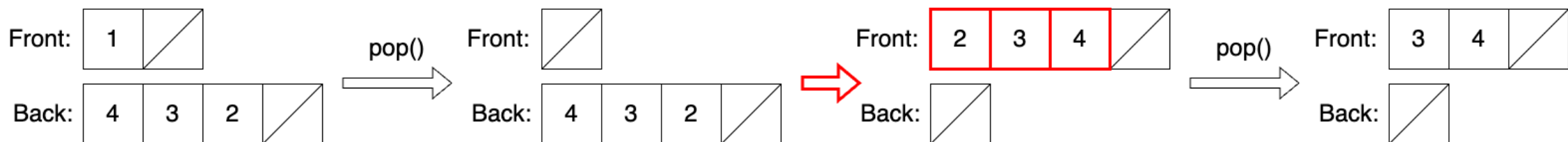


泛型数据结构：队列

- 我们用两个堆栈模拟队列

```
1. struct Queue[T] {
2.     front : Stack[T] // 负责取出操作
3.     back  : Stack[T] // 负责存储操作
4. }
```

- 当添加数据时，存入 `back`；当读取数据时，从 `front` 中取出
- 操作后，若 `front` 为空，则通过反转队列，将 `back` 转为 `front`
 - 确保若队列非空，则 `front` 非空
 - 队列反转的开销将在多次读取中平摊



泛型数据类型：队列

```
1. struct Queue[T] {
2.   front : Stack[T]
3.   back  : Stack[T]
4. }
5. fn[T] Queue::empty() -> Queue[T] { {front : Empty, back : Empty} }
6. fn[T] Queue::push(self : Queue[T], value : T) -> Queue[T] { // 将元素存入队尾
7.   normalize({ ..self, back : self.back.push(value)}) // 通过定义第一个参数为self, 我们可以用xxx.f()
8. }
9. fn[T] Queue::pop(self : Queue[T]) -> (Option[T], Queue[T]) { // 取出第一个元素
10.   match self.front {
11.     Empty => (None, self)
12.     NonEmpty(top, rest) => (Some(top), normalize({ ..self, front: rest}))
13.   }
14. }
15. fn[T] Queue::normalize(self : Queue[T]) -> Queue[T] { // 如果front为空, 反转back到front
16.   match self.front {
17.     Empty => { front: self.back.reverse(), back: Empty }
18.     _ => self
19.   }
20. }
21. fn[T] Stack::reverse(self : Stack[T]) -> Stack[T] { // 辅助函数: 反转堆栈
22.   // 省略实现
23. }
```

高阶函数

一些列表操作

- 求一个整数列表的和

```
1. fn sum(list : List[Int]) -> Int {  
2.     match list {  
3.         Nil => 0  
4.         Cons(hd, tl) => hd + sum(tl)  
5.     }  
6. }
```

- 求一个列表长度

```
1. fn[T] length(list : List[T]) -> Int {  
2.     match list {  
3.         Nil => 0  
4.         Cons(hd, tl) => 1 + length(tl)  
5.     }  
6. }
```


一些列表操作

- 我们发现它们有共通点

```
1. fn[A, B] func(list : List[A]) -> B {  
2.   match list {  
3.     Nil => b // b : B  
4.     Cons(hd, tl) => f(hd, func(tl)) // f : (A, B) -> B  
5.   }  
6. }
```

- 在之前的例子中
 - 在求和中, `b` 为0, `f` 为 `fn f(a, b) { a + b }`
 - 在求长度中, `b` 为0, `f` 为 `fn f(a, b) { 1 + b }`
- 如何重用这个结构呢?

函数是一等公民

- 在月兔中，函数是一等公民。这就意味着，我们可以把函数作为参数传递，也可以将函数作为计算结果存储：
 - 以刚才的结构为例，函数可以作为参数传递

```
1. fn[A, B] fold_right(list : List[A], f : (A, B) -> B, b : B) -> B {  
2.   match list {  
3.     Nil => b  
4.     Cons(hd, tl) => f(hd, fold_right(tl, f, b))  
5.   }  
6. }
```

- 高阶函数：接受函数作为参数或以函数作为运算结果的函数

函数是一等公民

- 重复一个函数的运算

```
1. fn[A] repeat(f : (A) -> A) -> (A) -> A { // repeat的类型是((A) -> A) -> (A) -> A
2.   fn (a) { f(f(a)) }
3. } // 函数作为计算的结果
4.
5. fn plus_one(i : Int) -> Int { i + 1 }
6. fn plus_two(i : Int) -> Int { i + 2 }
7.
8. let add_two : (Int) -> Int = repeat(plus_one) // 存储函数
9.
10. test {
11.   assert_eq(add_two(2), plus_two(2)) // 两者皆为4
12. }
```

高阶函数的化简

```
let add_two : (Int) -> Int = repeat(plus_one)
```

```
  repeat(plus_one)
```

$\mapsto$ `fn (a) { plus_one(plus_one(a)) }`

替换表达式中的标识符

```
let x : Int = add_two(2)
```

```
  add_two(2)
```

$\mapsto$ `plus_one(plus_one(2))`

替换表达式中的标识符

$\mapsto$ `plus_one(2) + 1`

替换表达式中的标识符

$\mapsto$ `(2 + 1) + 1`

替换表达式中的标识符

$\mapsto$ `3 + 1` $\mapsto$ `4`

函数的类型

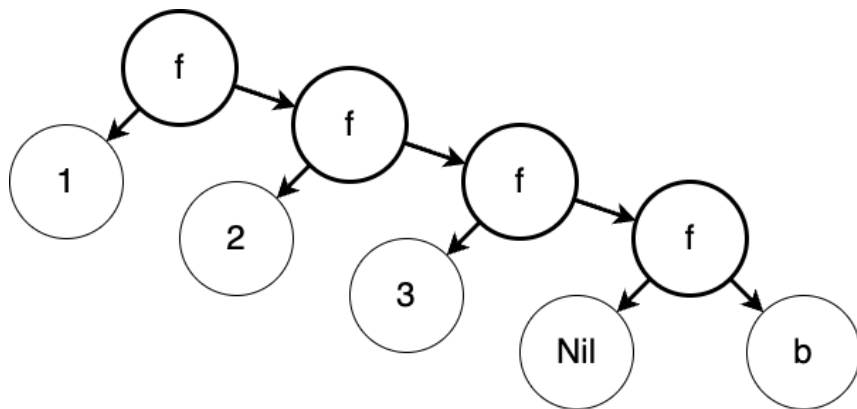
- 函数的类型为 $(t_{\text{in}}) \rightarrow t_{\text{out}}$ ，如
 - `(Int) -> Int` 整数到整数
 - `(Int) -> (Int) -> Int` 整数到函数
 - `(Int) -> ((Int) -> Int)` 同上
 - `((Int) -> Int) -> Int` 函数到整数

高阶函数的应用：列表折叠

- 我们刚才已经看到了列表折叠的一种可能性

```
1. fn[A, B] fold_right(list : List[A], f : (A, B) -> B, b : B) -> B {  
2.   match list {  
3.     Nil => b  
4.     Cons(hd, tl) => f(hd, fold_right(tl, f, b))  
5.   }  
6. }
```

- 这种折叠从右向左构建，因此被称为 `fold_right`

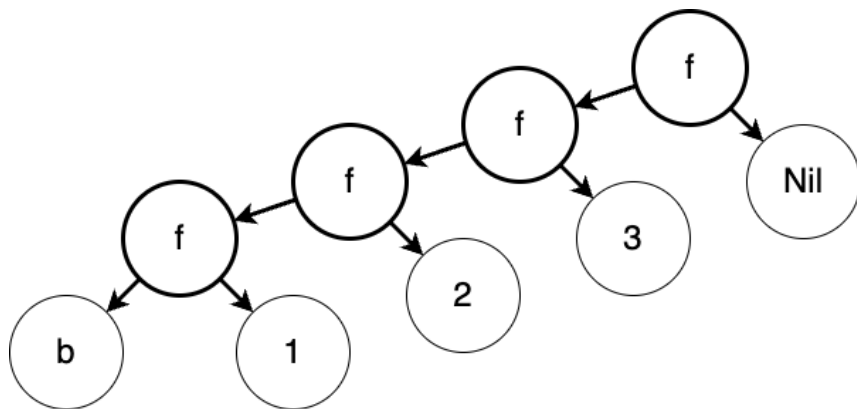


高阶函数的应用：列表折叠

- 我们也可以从另一个方向进行折叠

```
1. fn[A, B] fold_left(list : List[A], f : (B, A) -> B, b : B) -> B {  
2.   match list {  
3.     Nil => b  
4.     Cons(hd, tl) => fold_left(tl, f, f(b, hd))  
5.   }  
6. }
```

- 这种折叠从左向右构建，因此被称为 `fold_left`



高阶函数的应用：列表映射

- 一个常见的操作是对列表中的每一个元素进行映射
 - 例如，从个人信息列表中获得姓名列表
 - `struct PersonalInfo { name : String; age : Int }`

```
1. fn[A, B] map(self : List[A], f : (A) -> B) -> List[B] {  
2.     match list {  
3.         Nil => Nil  
4.         Cons(hd, tl) => Cons(f(hd), map(tl, f))  
5.     }  
6. }  
7. let infos : List[PersonalInfo] = { ... }  
8. let names : List[String] = infos.map(fn (info) { info.name })
```


高阶函数的应用：列表映射

- 事实上，我们还可以用 `fold_right` 来实现 `map` 函数

```
1. fn[A, B] map(list : List[A], f : (A) -> B) -> List[B] {  
2.   fold_right(list, fn(value, cumulator) { Cons(f(value), cumulator) }, Nil)  
3. }
```

- 思考题：如何用 `fold_right` 来实现 `fold_left` ?

二叉搜索树

- 我们定义一个更一般的二叉搜索树，允许存放任意类型的数据

```
1. // 我们利用泛型定义数据结构
2. enum Tree[T] {
3.     Empty
4.     Node(T, Tree[T], Tree[T])
5. }
6.
7. // 我们需要一个比较函数来比较值的大小以了解顺序
8. // 负数表示小于，0表示等于，正数表示大于
9. fn[T] insert(self : Tree[T], value : T, compare : (T, T) -> Int) -> Tree[T] { ... }
10. fn[T] delete(self : Tree[T], value : T, compare : (T, T) -> Int) -> Tree[T] { ... }
```

为什么需要迭代器

- 很多数据类型都包含可以按顺序访问的元素，例如列表、树（前/中/后序）、队列等
- 统一的遍历接口，无需关心底层数据结构
- 惰性，不会一次性访问所有数据，而是按需产生数据

获取大于100的前十个完全平方数

```
1. test {
2.   let naturals = Iter::new(fn(yield_) {
3.     fn go(n) {
4.       guard yield_(n) is IterContinue else { IterEnd }
5.       go(n + 1)
6.     }
7.
8.     go(0)
9.   })
10.  let n = naturals
11.    .map(fn(x) { x * x })           // 所有自然数的平方的迭代器
12.    .filter(fn(x) { x > 100 })     // 筛选出大于 100 的值
13.    .take(10)                      // 取前 10 个结果
14.  inspect(n, content="[121, 144, 169, 196, 225, 256, 289, 324, 361, 400]")
15. }
```

迭代器

- 迭代器是一个对象，用于遍历序列中的每一项，并决定何时结束
- 迭代器类型 `Iter[T]` 用于遍历元素类型为 `T` 的序列

```
1. let list123 = Cons(1, Cons(2, Cons(3, Nil)))  
2. let iter : Iter[Int] = list123.iter()  
3. iter.each(println) // println 会打印数字并加上一个换行符
```

会打印出：

```
1  
2  
3
```

消费迭代器

- 迭代器有多种消费方式

```
1. fn[T] Iter::each(self : Iter[T], f : (T) -> Unit) -> Unit { ... }
2. fn[T] Iter::count(self : Iter[T]) -> Int
3. fn[T] Iter::head(self : Iter[T]) -> Option[T]
4. fn[T] Iter::nth(self : Iter[T], n : Int) -> Option[T]
```

- ```
1. test {
2. let iter : Iter[Int] = list123.iter()
3. inspect(iter.count(), content="3")
4. inspect(iter.head(), content="Some(1)")
5. inspect(iter.nth(1), content="Some(2)")
6. }
```

- 迭代器是惰性的：不访问到某个元素时，不会为这个元素分配内存

# 转换迭代器

- 迭代器的常见用法是经过多轮转换，最终得到需要的结果
- 以下函数使得迭代器转换为新的迭代器：

```
1. fn[T, R] Iter::map(self : Iter[T], f : (T) -> R) -> Iter[R] { ... }
2. fn[T] Iter::filter(self : Iter[T], p : (T) -> Bool) -> Iter[T] { ... }
3. fn[T] Iter::take(self : Iter[T], n : Int) -> Iter[T] { ... }
4. fn[T] Iter::concat(self : Iter[T], other : Iter[T]) -> Iter[T] { ... }
```

- 利用映射转换迭代器：

```
1. test {
2. let iter = list123.iter().map(fn (x) { x + 1 })
3. inspect(iter, content="[2, 3, 4]") // 迭代器以数组的形式展现
4. }
```

# 从数据结构生成迭代器

- 如何为数据结构生成迭代器呢？

本质上，迭代器类型 `Iter[T]` 即是 `((T) -> IterResult) -> IterResult`

- `IterResult` 有两种情形：
  - `IterContinue` 代表继续迭代；
  - `IterEnd` 代表迭代结束。
- 第一个 `IterResult` 表示下游迭代过程是否结束，第二个 `IterResult` 表示自身迭代过程是否结束
- 假设 `f` 的参数名为 `yield_`。迭代发生时，需要将元素一个个“喂”给 `yield_` 函数，并根据调用结果和内部状态决定继续迭代还是终止迭代



# 列表迭代器

```
1. fn[T] List::iter(self : List[T]) -> Iter[T] {
2. fn go(list : List[T], yield_ : (T) -> IterResult) -> IterResult {
3. match list {
4. Nil => IterContinue
5. Cons(head, tail) => {
6. guard yield_(head) is IterContinue else { IterEnd }
7. go(tail, yield_)
8. }
9. }
10. }
11.
12. Iter::new(fn (yield_) { go(self, yield_) })
13. }
```

- 迭代器的构造函数:

```
fn[T] Iter::new(f : ((T) -> IterResult) -> IterResult) -> Iter[T]
```

# 计数器和惰性

- 如下迭代器可以生成无穷的自然数

```
1. test {
2. let naturals = Iter::new(fn(yield_) {
3. fn go(n) {
4. guard yield_(n) is IterContinue else { IterEnd }
5. go(n + 1)
6. }
7.
8. go(0)
9. })
10.
11. inspect(naturals.nth(0), content="Some(0)")
12. inspect(naturals.nth(10), content="Some(10)")
13. inspect(naturals.nth(100), content="Some(100)")
14. }
```

## 获取大于100的前十个完全平方数

- 第 0 次迭代

```
1. let n = naturals // yield_(0), IterContinue
2.
3. .map(fn(x) { x * x }) // yield_(0), IterContinue
4.
5. .filter(fn(x) { x > 100 }) // IterContinue
6.
7. .take(10) //
```

## 获取大于100的前十个完全平方数

- 第 11 次迭代

```
1. let n = naturals // yield_(11), IterContinue
2.
3. .map(fn(x) { x * x }) // yield_(121), IterContinue
4.
5. .filter(fn(x) { x > 100 }) // yield_(121), IterContinue
6.
7. .take(10) // yield_(121), IterContinue
```

## 获取大于100的前十个完全平方数

- 第 20 次迭代

```
1. let n = naturals // yield_(20), IterEnd
2.
3. .map(fn(x) { x * x }) // yield_(400), IterEnd
4.
5. .filter(fn(x) { x > 100 }) // yield_(400), IterEnd
6.
7. .take(10) // yield_(400), IterEnd
```

# 总结

- 本章节我们学习了
  - 泛型和函数是一等公民的概念
  - 数据结构堆栈与队列的实现
  - 用于迭代数据结构的迭代器
- 推荐阅读
  - *Software Foundations* 第四章 或
  - *Programming Language Foundations in Agda* 第十章